

Malena Mahoney

Scav-a-Snap

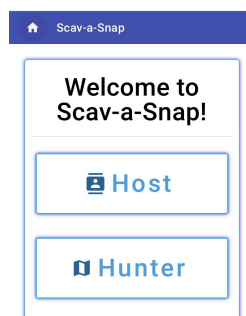
Overview

Scav-a-Snap is a photo scavenger hunt app for anyone looking for an exciting team-building experience. It provides all ages with enjoyable lasting memories.

Scav-a-Snap allows users to create and host personalized hunts for a group of hunters. A hunt consists of a title, description, estimated time, and tasks. Once a hunt is created, the host can edit the title, description, and estimated time. There is also the functionality of adding and removing tasks. When a host creates a hunt, that hunt is a part of their profile and will be available for them to use as many times as they want and can also be deleted if desired.

Once a host chooses the number of teams that will be participating, they may begin a hunt. A started hunt will have a unique access code that hunters may use to join the hunt. By entering the access code, a hunter will be able to create a team and add all members to the team. Only one device can be used for each team of hunters.

After joining the hunt, each team can complete tasks by capturing pictures of specific items or locations. The host can track the progress the teams are making and have the option to end the hunt whenever they would like. When the hunt ends, the host can view the hunt at any time by selecting it on their profile page. They can view all the photos submitted for each task with captions labeling what team took what photo.



Technologies Used:

- Angular 17
- Javalin
- Angular Material
- Mongo Database
- Jasmine and Karma
- GitHub
- GitHub Projects
- Java 17
- TypeScript
- HTML
- CSS
- Visual Studio Code

Agile Development

This project was built using an agile approach. In agile development, communication with the customer is the key to a successful software project. During each iteration, our team made user stories to present to the customer, actively seeking their input to guide our development of the project. At the end of each iteration, there was a showcase to demonstrate our completed work and gather additional feedback, which was essential for guiding our next steps.

Along with customer interaction, effective communication, collaboration, and adaptability are paramount. Regular team meetings kept us informed about project status and individual responsibilities. Each iteration presented me with a variety of roles and leadership positions, all aimed at delivering optimal results for our customers. Additionally, the use of GitHub for organization proved invaluable. It enabled us to track progress, manage tasks, and maintain a clear project overview, fostering accountability, and seamless collaboration.

In conclusion, the Agile development approach, with its emphasis on customer interaction, team collaboration, adaptability, and organization, was instrumental in the successful development of Scav-a-Snap. It led to a high-quality product that met the customer's needs and expectations and

provided a rewarding and educational experience for the team. I look forward to applying the lessons learned in this project to future endeavors.

Personal Contributions

This project was split into 3 iterations. The first iteration had a total of 297 commits, 76 of them being mine. Iteration 2 had 675 total commits, with me producing 148 of them. The final iteration had 1041 commits and I was responsible for 230 of those. I did a lot of front-end development and testing, with a bit of back-end development and MongoDB implementation.

Basic Host Features

I implemented several features for the host, including delete hunt/task, editing hunt, and creating team functionalities.

The 'deleteHunt/Task' feature allows users to remove hunts or tasks they no longer need. DeleteHunt/Task takes the ID of a hunt/task object on the client side and sends a delete request to the server. The method in the server then deletes the hunt/task from the corresponding MongoDB collection using the given ID. Users can access the 'deleteHunt' and 'deleteTask' buttons directly from the hunt details page. After clicking the 'deleteHunt' or 'deleteTask' button, a confirmation dialog appears, asking the user to confirm their action. Once confirmed, the user is automatically redirected to their host profile page, and the deleted hunt/task no longer appears in the hunt details. If the user does not confirm, the hunt will remain. After the delete task button is selected and the user confirms the deletion, the task is automatically removed from the hunt details page. If the specified hunt/task ID is invalid or if an error occurs during deletion, an appropriate error message is displayed.

For editing hunts, there is an edit icon on the hunt details page that allows users to change the hunt title, description, and estimated time. When the icon is clicked, a form will pop up with the current title, description, and estimated time pre-filled into the form fields. Users can modify the form fields, and the edit will be complete once they select the 'Complete Edit' button. The hunt's information will be updated instantly. I used Angular FormBuilder to develop this form and implemented the 'updateHunt' method on the server side. The 'updateHunt' method uses a PUT

request to retrieve a hunt ID. With the given ID, it updates the hunt with the matching ID in the MongoDB database.

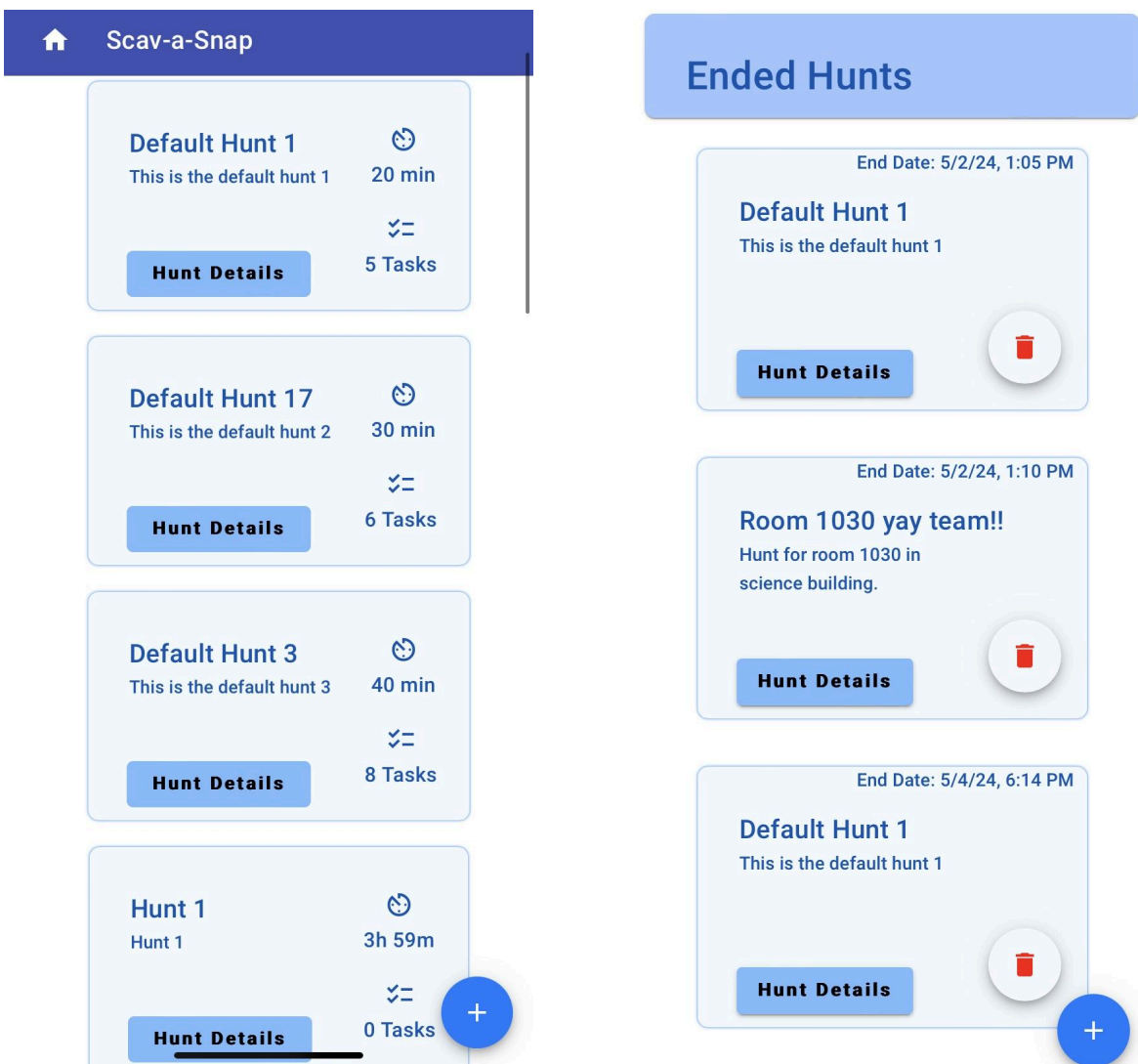
Teams are formed by the hunters. When a hunter joins a hunt, they are directed to the team creation form. The form includes two main fields: one for the team name and another for team members. Users can add members by selecting 'Add Member,' and they can remove members by choosing 'Remove Member.' The team cannot be created until all form fields are filled out. If a member field is left empty, it must be removed. Additionally, error messages are provided to guide users if any issues arise during form submission. Once the form is complete, users can select 'CREATE TEAM.' To implement this functionality, I introduced a new object called 'TeamHunt' and created the client-side method 'createTeam.' This method accepts a partial 'TeamHunt' object, returns a string observable, and sends a post request to the server.

The image shows a mobile app interface for 'Scav-a-Snap'. At the top is a dark blue header with a home icon and the text 'Scav-a-Snap'. Below the header, there's a card for 'Default Hunt 1' with a clock icon and '20 min'. A white modal titled 'Edit Hunt' is open in the foreground. It contains three input fields: 'Hunt Name*' with the value 'Default Hunt 1', 'Edit Description*' with the value 'This is the default hunt 1', and 'Edit Estimated Time*' with the value '20'. Below the modal, there's a list of tasks: '2. Take a picture of a Stop sign', '3. Take a picture of a bird', '4. Take a picture of a dog', and '5. Take a picture of a restaurant'. Each task has a red trash icon to its right.

The image shows a mobile app interface for 'Scav-a-Snap'. At the top is a dark blue header with a home icon and the text 'Scav-a-Snap'. Below the header, there's a white card titled 'Create Team'. It contains two input fields: 'Team Name*' and 'Team Member*'. Below these fields are two buttons: 'Remove Member' and 'Add Member'. At the bottom right of the card are two buttons: 'CANCEL' and 'CREATE TEAM'.

Display

I significantly contributed to the display of Scav-a-Snap. Most of my efforts were on the host side rather than the hunter side. I coded several Angular Materials components, including cards, buttons, form fields, dialogs, snackbars, and icons. Ensuring proper display and functionality required a solid understanding of HTML, TypeScript, and SCSS. The biggest challenge I overcame was making the display look good on any size screen.



Unit Testing

For this project, I contributed to the application's reliability and functionality by conducting thorough client-side testing using Jasmine and Karma. My focus was on unit testing, which allowed me to verify various features. Specifically, I tested the `hostService` using the `MockHostService` class, covering essential methods such as `deleteHunt`, `deleteTask`, `getHunt`, and `createTeam`. By testing these functionalities, I ensured they performed correctly across different scenarios. Additionally, I contributed by writing tests for other components, including `HostProfileComponent`, `CreateTeamComponent`, `HuntProfileComponent`, and `EditHuntComponent`.

Karma v 6.4.3 - connected; test: complete;

Chrome 124.0.0.0 (Windows 10) is idle

Jasmine 4.6.0

153 specs, 0 failures, randomized with seed 12885

```
HuntProfileComponent
  • should navigate to correct hunt when the id parameter changes
  • should navigate to a specific hunt profile
  • should create the component
  • should set error data on observable error
  • should have 'null' for the hunt for a bad ID

DeleteHunt() and StartHunt()
  • should call deleteTask on HuntService when deleteTask is called in HuntProfileComponent
  • should delete a hunt and navigate to /hosts
  • should delete a task and remove it from the tasks array
  • should call deleteTask when onHuntDeleteClick is confirmed
  • should call deleteTask when onTaskDeleteClick is confirmed
  • should call deleteHunt on HuntService when deleteHunt is called in HuntProfileComponent

When onHuntDeleted() is called
  • should remove the hunt with the given id from serverEndedHunts when onHuntDeleted is called

AddTaskComponent#submitForm()
  • should call hostService.addTask and update completeHunt
  • should return false when the control is invalid but not dirty or touched
  • should return false when the control is valid
  • should call addTask() and handle error response
  • should return true when the control is invalid and either dirty or touched

StartHuntComponent
  • should call endHunt() when onEndHuntClick is confirmed
  • should handle error when ending the hunt fails
  • should end the hunt successfully
  • should calculate tasks progress accurately
  • should create the component
  • should have 'null' for the started hunt for a bad accessCode
  • should navigate to a specific started hunt
  • should set beginHunt to true when beginHunt() is called
  • should navigate to correct startedHunt when the accessCode parameter changes
  • should set error data on observable error

HostService
  Deleting an ended hunt using 'deleteEndedHunt()'
    • talks to the right endpoint and is called once
  Ending a hunt using 'endStartedHunt()'
    • calls api/startedHunts/id with the correct ID
  Submitting a photo using 'submitPhoto()'
    • talks to the right endpoint and is called once
  When createTeam() is given an partial TeamHunt
    • talks to the right endpoint and is called once
    • should update hunt
  Adding a task using 'addTask()'
    • talks to the right endpoint and is called once
  Adding a hunt using 'addHunt()'
    • talks to the right endpoint and is called once
  When getHunts() is called
    • calls 'api/hosts/575'
  When getTeamHunt() is given an ID
    • calls api/teams/id with the correct ID
  When getStartedHunt() is given an ID
    • calls api/startedHunts/id with the correct ID
  When getEndedHuntById() is given an ID
    • calls api/endedHunts/id with the correct ID
  When getHuntById() is given an ID
    • calls api/hunts/id with the correct ID
  Deleting a hunt using 'deleteHunt()'
    • talks to the right endpoint and is called once
  When getEndedHunts() is called
    • calls api/hosts/{id}/endedHunts
```